

Week 15: Perceptron & Multilayer Perceptron (MLP)

Submission Guidelines:

1. Each task must properly show the output. DO NOT Clear the outputs after running.
2. **Download the .ipynb file** from Jupyter Notebook.
3. **Upload the .ipynb** to GCR.
4. **Failure to follow the submission guidelines will result in a deduction of 50% of the obtained marks.**

Task 1: Implementation of Single-Layer Perceptron for Linearly Separable Classification

Objective

To design and implement a single-layer perceptron model **for binary classification** of linearly separable datasets and analyze its **convergence** behavior under **different learning rates**.

Problem Statement

Develop a perceptron-based classifier to model logical decision boundaries using synthetic Boolean datasets.

Dataset Specification

- Dataset Type: Synthetic Binary Classification Dataset
- Input Features: 2-dimensional binary vectors
- Classes:
 - Class 0: (0,0), (0,1)
 - Class 1: (1,0), (1,1) (*for OR case*)
- Additionally test:
 - AND logic dataset

Implementation Requirements

- Implement perceptron from scratch using Python (NumPy only)
- Initialize weights randomly
- Use step activation function
- Implement:
 - Weight update rule
 - Bias adjustment

- Train model until convergence or max epochs reached

Evaluation Criteria

- Decision boundary visualization
- Epochs required for convergence
- Effect of learning rate variation (0.1, 0.01, 0.5)

Task 2: Analytical Study of Activation Functions and Decision Boundaries

Objective

To critically analyze the effect of activation functions on perceptron output and geometric decision boundaries in feature space.

Problem Statement

Evaluate and compare the behavior of step, sigmoid, and ReLU activation functions in binary classification scenarios.

Dataset Specification

- Synthetic 2D dataset with Gaussian distributed clusters
- Two separable classes with controlled overlap

Implementation Requirements

- Implement perceptron model with:
 - Step activation
 - Sigmoid activation
 - ReLU activation
- Plot:
 - Decision boundaries
 - Output probability surfaces (for sigmoid)

Evaluation Criteria

- Graphical correctness of decision boundaries
- Comparative analysis report (bias, variance, sensitivity)

Task 3: XOR Problem Solving Using Multilayer Perceptron (MLP)

Objective:

To implement a Multi-Layer Perceptron (MLP) from scratch and demonstrate its ability to learn non-linearly separable patterns using the XOR problem.

Dataset Description:

The XOR dataset consists of four input-output pairs:

Input (x_1, x_2)	Output
(0, 0)	0
(0, 1)	1
(1, 0)	1
(1, 1)	0

This dataset is not linearly separable and cannot be solved using a single-layer perceptron.

Implementation Requirements:

- Implement the neural network **from scratch using NumPy only** (no ML libraries).
- Use:
 - One hidden layer with **at least 4 neurons**
 - Sigmoid activation function for both hidden and output layers
- Initialize weights randomly and biases as zeros.
- Train the network using **forward propagation and backpropagation**.
- Use **Mean Squared Error (MSE)** as the loss function.
- Train the model for **at least 10,000 epochs**.
- Print the loss value at regular intervals (e.g., every 1000 epochs).
- After training, output the predicted values for all input combinations.

Expected Outcome:

- The model should converge such that:
 - Outputs for class 0 ≈ 0
 - Outputs for class 1 ≈ 1
- If output is near 0.5, it indicates inadequate learning or incorrect implementation.

Task 4: Backpropagation Algorithm – Step-by-Step Computational Simulation

Objective

To mathematically and computationally implement the backpropagation learning algorithm for a small neural network and validate gradient updates.

Problem Statement

Simulate a three-layer neural network and manually compute forward and backward passes for a single training iteration.

Dataset Specification

- Input: Small synthetic dataset
 - $X = \{(0,0), (0,1), (1,0), (1,1)\}$
- Output:
 - Binary classification target

Implementation Requirements

- Construct a 2-2-1 MLP architecture
- Perform:
 - Weight initialization (small random values)
 - Forward propagation
 - Error computation
 - Backpropagation of error gradients
- Manually verify at least one iteration of:
 - Hidden layer delta computation
 - Output layer weight adjustment

Evaluation Criteria

- Mathematical correctness of gradient computation
- Code correctness of forward/backward propagation
- Consistency between manual and program output

Task 5: Real-World Classification Using MLP (Scikit-learn Implementation)

Objective:

To train and evaluate a Multi-Layer Perceptron classifier on a real-world dataset and assess its performance using appropriate evaluation metrics.

Dataset Description:

Use the Breast Cancer Wisconsin dataset available in `sklearn.datasets`.

- Features: 30 numerical attributes computed from digitized images of breast masses
- Target: Binary classification (malignant = 0, benign = 1)

Implementation Requirements:

- Load the dataset using `sklearn.datasets.load_breast_cancer`.
- Perform **feature standardization** using `StandardScaler`.
- Split the dataset into training and testing sets using:
 - `test_size = 0.2`
 - `random_state = 42` (for reproducibility)
- Train an `MLPClassifier` with:
 - At least **two hidden layers**
 - Maximum iterations ≥ 500
- Evaluate the trained model on the test set and report:
 - Confusion Matrix
 - Precision, Recall, and F1-score
- Additionally, perform **5-fold cross-validation** on the entire dataset and report:
 - Individual fold accuracies
 - Mean cross-validation accuracy

Expected Outcome:

- The model should achieve high classification performance (typically >95% accuracy).
- Cross-validation results should be consistent across folds, indicating model stability.

Task 6: End-to-End Neural Network Training Pipeline with Checkpointing and Early Stopping

Objective

To design and implement a complete neural network training pipeline incorporating iterative optimization, model checkpoint preservation, stopping criteria, and best-model restoration for robust supervised learning.

Problem Statement

Develop a fully functional feedforward neural network training system for classification. The system must automate the complete model development lifecycle including preprocessing, training, validation monitoring, checkpoint saving, stopping conditions, and final testing.

Dataset Specification

Use the following benchmark dataset:

Fashion MNIST

- Samples: 70,000 grayscale images
- Image Size: 28×28
- Classes: 10 clothing categories

Implementation Requirements

1. Data Processing Pipeline

- Load dataset using sklearn / keras.datasets
- Handle missing values if applicable
- Normalize or standardize input features
- Split dataset into:
 - Training Set (70%)
 - Validation Set (15%)
 - Test Set (15%)

2. Neural Network Architecture

Construct a Multilayer Perceptron containing:

- Input Layer (based on feature count)
- Minimum one hidden layer (32 / 64 neurons)

- Output Layer:
 - Softmax for multiclass
 - Sigmoid for binary classification

Recommended activations:

- ReLU in hidden layers

3. Training Configuration

Use:

- Loss Function:
 - CrossEntropyLoss / BinaryCrossEntropy
- Optimizer:
 - SGD / Adam
- Batch Size: 16 / 32
- Epoch Limit: 100

4. Mandatory Checkpointing Mechanism

During training:

- Save model weights after each epoch if validation loss improves
- Filename format:

best_model_epoch_05.pth

or

best_model.h5

Store:

- Epoch number
- Model state_dict / weights
- Optimizer state
- Validation accuracy
- Validation loss

5. Early Stopping / Stopping Criteria

Terminate training when any of the following occurs:

Criterion A: Patience-Based Early Stopping

If validation loss does not improve for **10 consecutive epochs**

Criterion B: Accuracy Threshold

Stop if validation accuracy exceeds **95%**

Criterion C: Maximum Epoch Limit

Training reaches 100 epochs

6. Best Model Restoration

After training:

- Reload best saved checkpoint
- Evaluate only best model on test set

7. Performance Visualization

Plot:

- Training Loss vs Validation Loss
- Training Accuracy vs Validation Accuracy